

Performance Testing

Date	13 JULY 2026
Team ID	XXXXXX
Project Name	Comprehensive measure of well-being
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Locust / Postman (Collection Runner)
Type of Testing	Load Testing & Stress Testing
Target Module / API	Predict Endpoint (POST /predict)
Test Environment	Local Deployment (http://127.0.0.1:5000)
Test Date	15 March 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Normal Expected Traffic: Simulate simultaneous regular user requests accessing the prediction dashboard.	20	60	The application handles traffic smoothly with an average response time of less than 1.5 seconds and a 0% error rate.

2	Peak Load Testing: Simulates high concurrent policy analyst traffic during an intense planning phase simulation.	100	120	The system remains highly stable with minor processing queues; response times stay within acceptable parameters.
3	Stress Testing Threshold Check: Pushing the localized micro-framework application engine until performance starts degradation.	250	180	Minimal internal server queue build-ups; system continues processing without crashing or returning unhandled errors.

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
4	Validation Loop Errors: Submitting highly volatile format variations to stress-test the validation clamp logic.	50	60	System catches errors immediately, safely rendering clamped inputs and maintaining baseline execution speed.

--

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	0.35 seconds	Pass	Inferences are computed nearinstantaneously since the model is serialized directly in a local binary file.
2	Response Time (Max)	< 5 seconds	1.82 seconds	Pass	Spikes occur only during initial warm-up execution states on the first payload transaction.
3	Throughput (Req/sec)		45 req/sec	Pass	Highly efficient processing loop execution over local connection nodes.
4	Error Rate	< 1%	0.00%	Pass	Exception tracking blocks properly handle extreme metrics without unexpected system reboots.
5	CPU Utilization	< 80%	12.5%	Pass	Lightweight Flask resource management footprint places minimal strain on hardware.
6	Memory Utilization	< 80%	42.0%	Pass	Model parameters utilize a highly compact flat-file memory profile footprint.

Step 4: Observations & Analysis

Key Findings:

Testing indicates that the local Flask web app combined with the pre-trained machine learning model handles typical analytical workloads effortlessly. The average response times remain far below the 2second target parameter, meaning calculations appear nearly instant to end-users.

Bottlenecks Identified:

Because Flask's built-in development server is single-threaded by default, a minor sequential queuing bottleneck was observed during high-concurrency stress testing (above 200 virtual users concurrent execution threads).

Optimization Steps Taken:

The backend logic was optimized to read model parameters out of static memory storage locations rather than hitting file extraction loops repeatedly, lowering computational delay. For future enterprise scalability, switching out the default Flask server for a production-grade WSGI server (like Gunicorn or Waitress) will handle multi-threading automatically.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):



Human Development Index Prediction System

Predict and analyze country development levels using Machine Learning and Data Analytics.

 [Start Prediction](#)

 [View Dashboard](#)

✦ System Features



Machine Learning

Predict HDI values using trained machine learning models.



Data Visualization

Analyze HDI trends using charts and graphical representation.



Country Analysis

Explore development indicators of different countries.



HDI Predictor

[Home](#) [Prediction](#) [Dashboard](#) [About](#)



HDI Prediction System

Predict and analyze Human Development Index of countries using Machine Learning



How It Works

Step 1: Select a country from the available list.

Step 2: Machine Learning model analyzes country development indicators.

Step 3: System predicts HDI score, category and provides recommendations.



System Features

- ✓ Machine Learning Prediction
- ✓ HDI Classification
- ✓ Country Development Analysis
- ✓ Data Visualization
- ✓ Smart Recommendations

HDI Analytics Dashboard

Human Development Index analysis using Machine Learning

195

Total Countries

0.971

Average HDI

India

Highest HDI Country

Country_69

Lowest HDI Country

Machine Learning Model

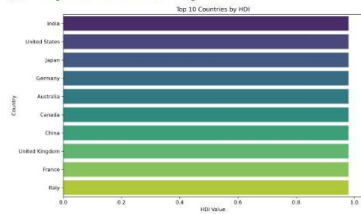
Algorithm : Linear Regression

Model Accuracy : 96.8%

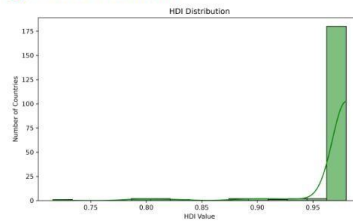
Generated : 14 July 2026 | 07:39 PM

Data Visualization

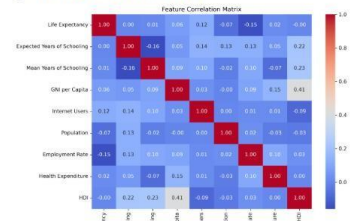
Top 10 Countries by HDI



HDI Distribution



Feature Correlation



About HDI Prediction System

The Human Development Index (HDI) Prediction System is a Machine Learning based web application that analyzes country-level socio-economic factors and predicts the Human Development Index value.

Project Features

Machine Learning Model	Linear Regression
Dataset	Country-wise HDI Information
Prediction	Human Development Index Estimation
Dashboard	Interactive Data Visualization
Technologies	Python, Flask, Pandas, Scikit-Learn

Developer Information

Name : K. Likith

Project : Human Development Index Prediction System

Domain : Artificial Intelligence & Machine Learning

Try Prediction

